Fintech 545 Midterm

Kurt Ma | October 19, 2026

Conventions: Arithmetic daily returns; VaR and expected shortfall are positive losses. Dollar risk is one-day risk computed from last observed portfolio values.

AI disclaimer:

Where did I use ai:

1. Organize code (example prompt: break down the setup cell. make each subquestion has its own code belonging to it.)
2. Implementating stuff from class (example prompt: implement the higham's algorithm.)

check prompt.txt for a detailed prompt breakdown (mostly used in question 1 and 3).

In [1]:
```
import numpy as np
import pandas as pd
from IPython.display import display
```

Question 1 — Pairwise Correlation with Missing Data

(a) Pairwise correlation matrix

For each pair (x_i, x_j), drop any row where either value is missing, then compute the Pearson correlation. Different pairs use different sample sizes, so this matrix need not be positive semidefinite.

In [2]:
```
# Problem 1(a)
p1 = pd.read_csv("problem1.csv")

print("Shape:", p1.shape, " | missing cells:", int(p1.isna().sum().sum
      " | complete rows:", int(p1.dropna().shape[0]))
display(pd.DataFrame({"observed": p1.notna().sum(), "missing": p1.isna
```

```
# n_ij = number of days both series are observed
p1_counts = pd.DataFrame(
    p1.notna().astype(int).T @ p1.notna().astype(int),
    index=p1.columns, columns=p1.columns,
)
print("Pairwise overlap counts n_ij:")
display(p1_counts)

# Pairwise Pearson: each pair uses only jointly observed rows (pandas
p1_corr = p1.corr()
print("Pairwise correlation matrix:")
display(p1_corr.round(6))
```

```
Shape: (50, 5)  | missing cells: 49  | complete rows: 18
```

	observed	missing
x1	41	9
x2	41	9
x3	36	14
x4	41	9
x5	42	8

```
Pairwise overlap counts n_ij:
```

	x1	x2	x3	x4	x5
x1	41	32	31	36	34
x2	32	41	28	34	35
x3	31	28	36	30	30
x4	36	34	30	41	34
x5	34	35	30	34	42

```
Pairwise correlation matrix:
```

	x1	x2	x3	x4	x5
x1	1.000000	0.332836	0.418462	0.255285	0.732272
x2	0.332836	1.000000	0.251035	0.287709	0.787538
x3	0.418462	0.251035	1.000000	0.483580	0.684257
x4	0.255285	0.287709	0.483580	1.000000	0.782632
x5	0.732272	0.787538	0.684257	0.782632	1.000000

Each pair uses a different overlap (28 to 36 joint observations; diagonals are the 36–42 observed days of that series).

(b) Variance of each series

This is the variance of the non-missing values of that series alone, not the variance over the pairwise overlap from (a).

In [3]:
```
# Problem 1(b)
p1_var = p1.var(ddof=1)
p1_stdev = np.sqrt(p1_var)
p1_var_table = pd.DataFrame({
    "n (all available)": p1.notna().sum(),
    "variance": p1_var,
    "std. deviation": p1_stdev,
})
print("Full-sample variance of each series:")
display(p1_var_table.round(9))
```

```
Full-sample variance of each series:
```

	n (all available)	variance	std. deviation
x1	41	0.000406	0.020151
x2	41	0.000480	0.021910
x3	36	0.000151	0.012293
x4	41	0.000842	0.029013
x5	42	0.000273	0.016527

Variances (9 d.p.): $x_1 = 0.000406074$, $x_2 = 0.000480066$, $x_3 = 0.000151110$, $x_4 = 0.000841739$, $x_5 = 0.000273150$. These are the diagonal of the covariance in (e).

(c) Is the correlation matrix PSD?

A symmetric matrix is positive semidefinite if and only if every eigenvalue is ≥ 0 (Week 02).

In [4]:
```
# Problem 1(c)
p1_corr_eigs = np.linalg.eigvalsh(p1_corr)
print("Eigenvalues of the pairwise correlation (ascending):")
print(np.array2string(p1_corr_eigs, precision=12))
print("Smallest eigenvalue:", p1_corr_eigs[0])
```

```
Eigenvalues of the pairwise correlation (ascending):
[-0.089278667697  0.461339798414  0.71908517244   0.823437223928
  3.085416472915]
Smallest eigenvalue: -0.08927866769735301
```

As the smallest eigenvalue is about $-0.0893 < 0$, the pairwise correlation is not PSD. Pairwise deletion is why this happens: each R_{ij} is estimated from a different sample, and the resulting matrix is not guaranteed to have a nonnegative spectrum.

(d) Higham's nearest PSD correlation

Higham's algorithm (Week 03, 3.6.2) alternates two projections, with Dykstra's correction:

1. the PSD cone (set negative eigenvalues to 0),
2. the unit-diagonal set (reset the diagonal to 1).

The input is the pairwise Pearson matrix from (a).

In [20]:
```
# Problem 1(d)
def _sym(a):
    a = np.asarray(a, dtype=float)
    return (a + a.T) / 2


def higham_nearest_correlation(R, tol=1e-12, max_iter=10000):
    """Nearest correlation matrix: Higham alternating projections + Dy
    y = _sym(R).copy()
    np.fill_diagonal(y, 1.0)
    delta_s = np.zeros_like(y)
    for iteration in range(1, max_iter + 1):
        r = y - delta_s
        vals, vecs = np.linalg.eigh(_sym(r))
        x = _sym((vecs * np.maximum(vals, 0.0)) @ vecs.T)  # PSD cone
        delta_s = x - r
        previous_y = y.copy()
        y = x.copy()
        np.fill_diagonal(y, 1.0)  # unit diagonal
        y = _sym(y)
        change = np.linalg.norm(y - previous_y, ord="fro")
        if change < tol and np.linalg.eigvalsh(y)[0] >= -tol:
            break
    else:
        raise RuntimeError("Higham did not converge")
    return y, iteration


p1_fix_corr, p1_iter = higham_nearest_correlation(p1_corr.to_numpy())
p1_fix_corr = pd.DataFrame(p1_fix_corr, index=p1.columns, columns=p1.c
p1_fix_eigs = np.linalg.eigvalsh(p1_fix_corr)

pd.set_option("display.width", 120)
```

```
pd.set_option("display.max_columns", 10)
print(f"Higham nearest PSD correlation R*  ({p1_iter} iterations)")
print(p1_fix_corr.round(6).to_string())
print()
shown_eigs = p1_fix_eigs.copy()
shown_eigs[np.abs(shown_eigs) < 1e-8] = 0.0
eig_table = pd.DataFrame(
    [shown_eigs],
    index=["eigenvalue"],
    columns=[f"e{k}" for k in range(1, len(shown_eigs) + 1)],
)
print("Eigenvalues of R*, smallest to largest")
print(eig_table.round(6).to_string())

delta = p1_fix_corr.to_numpy() - p1_corr.to_numpy()
np.fill_diagonal(delta, 0.0)
delta_df = pd.DataFrame(delta, index=p1.columns, columns=p1.columns)
print()
print("Entrywise change R* - R (diagonal is fixed at 1)")
print(delta_df.round(6).to_string())

abs_delta = np.abs(delta)
i, j = np.unravel_index(np.argmax(abs_delta), abs_delta.shape)
print()
print(
    f"Largest absolute change: {abs_delta[i, j]:.6f} at ({p1.columns[i
    f"{p1_corr.to_numpy()[i, j]:.6f} -> {p1_fix_corr.to_numpy()[i, j]:
)
```

```
Higham nearest PSD correlation R*  (36 iterations)
          x1        x2        x3        x4        x5
x1  1.000000  0.349452  0.425942  0.270782  0.694572
x2  0.349452  1.000000  0.260599  0.307523  0.739337
x3  0.425942  0.260599  1.000000  0.492500  0.662558
x4  0.270782  0.307523  0.492500  1.000000  0.737677
x5  0.694572  0.739337  0.662558  0.737677  1.000000

Eigenvalues of R*, smallest to largest
             e1        e2        e3        e4        e5
eigenvalue  0.0  0.450082  0.703476  0.806708  3.039734

Entrywise change R* - R (diagonal is fixed at 1)
          x1        x2        x3        x4        x5
x1  0.000000  0.016616  0.007480  0.015497 -0.037700
x2  0.016616  0.000000  0.009564  0.019814 -0.048201
x3  0.007480  0.009564  0.000000  0.008920 -0.021699
x4  0.015497  0.019814  0.008920  0.000000 -0.044955
x5 -0.037700 -0.048201 -0.021699 -0.044955  0.000000

Largest absolute change: 0.048201 at (x2, x5), 0.787538 -> 0.739337
```

R^* is PSD (smallest eigenvalue is numerically 0; the others are about 0.450, 0.703, 0.807, 3.040). The (x_2, x_5) entries change the most: $0.7875 \to 0.7393$

(absolute change 0.0482). Next are (x_4, x_5) and (x_1, x_5). All four x_5 correlations are pulled down; the other pairs move up slightly. The diagonal stays 1.

(e) Combine into a covariance matrix

Scale the repaired correlation by the full-sample standard deviations from (b):

$$\Sigma^* = DR^*D, \qquad D = \mathrm{diag}(\sigma_1, \ldots, \sigma_5).$$

The diagonal of Σ^* is exactly the variances in (b).

In [6]:
```
# Problem 1(e)
p1_cov_star = p1_fix_corr.to_numpy() * np.outer(p1_stdev.to_numpy(), p
p1_cov_star = pd.DataFrame(p1_cov_star, index=p1.columns, columns=p1.c

print("Combined covariance Σ* = D R* D:")
display(p1_cov_star.round(9))
print("Diagonal vs. full-sample variances from (b):")
display(pd.DataFrame({
    "diag(Σ*)": np.diag(p1_cov_star),
    "variance from (b)": p1_var,
}).round(12))
print("Σ* eigenvalues:", np.array2string(np.linalg.eigvalsh(p1_cov_sta
```

```
Combined covariance Σ* = D R* D:
```

	x1	x2	x3	x4	x5
x1	0.000406	0.000154	0.000106	0.000158	0.000231
x2	0.000154	0.000480	0.000070	0.000195	0.000268
x3	0.000106	0.000070	0.000151	0.000176	0.000135
x4	0.000158	0.000195	0.000176	0.000842	0.000354
x5	0.000231	0.000268	0.000135	0.000354	0.000273

```
Diagonal vs. full-sample variances from (b):
```

	diag(Σ*)	variance from (b)
x1	0.000406	0.000406
x2	0.000480	0.000480
x3	0.000151	0.000151
x4	0.000842	0.000842
x5	0.000273	0.000273

```
Σ* eigenvalues: [-1.202228639377e-16  9.469966423189e-05  2.91979485412
9e-04
  4.623009505664e-04  1.303158654112e-03]
```

The reported covariance uses the Higham correlation from (d) and the full-sample variances from (b). The diagonal is unchanged from (b); off-diagonals are $\rho^*_{ij}\sigma_i\sigma_j$. Σ^* is PSD because R^* is PSD and D is a positive diagonal.

Question 2 — VaR and ES for a Single Stock

XYZ is related to the market by

$$r_{XYZ} = \alpha + \beta r_{SPY} + \epsilon,$$

and the arithmetic returns of SPY and ϵ are normally distributed. Returns are arithmetic, $P_t/P_{t-1} - 1$. VaR and ES are positive losses.

(a) VaR

$$VaR_\alpha(x) = -F_x^{-1}(\alpha)$$

Fit α, β by OLS. Sample moments use $ddof = 1$, matching `Statistics.std`. Independence of the two normals gives

$$\mu = \alpha + \beta\mu_{SPY}, \qquad \sigma^2 = \beta^2\sigma^2_{SPY} + \sigma^2_\epsilon, \qquad r_{XYZ} \sim N(\mu, \sigma^2).$$

Then $F^{-1}(\alpha) = \mu + \sigma z_\alpha$ with $z_\alpha = \Phi^{-1}(\alpha)$, so the positive loss is $VaR_\alpha = -\mu - \sigma z_\alpha$. Report $100 \times VaR_\alpha$ at $\alpha = 0.05$ and $\alpha = 0.01$ (Week 04).

In [7]:
```
# Problem 2(a)
from scipy import stats

prices = pd.read_csv("problem2.csv", parse_dates=["Date"])
returns = prices[["SPY", "XYZ"]].pct_change().dropna()

y = returns["XYZ"].to_numpy()
x = returns["SPY"].to_numpy()
alpha, beta = np.linalg.lstsq(np.column_stack([np.ones(len(x)), x]), y
epsilon = y - (alpha + beta * x)

mu_spy = x.mean()
sigma_spy = x.std(ddof=1)
sigma_eps = epsilon.std(ddof=1)
mu_xyz = alpha + beta * mu_spy
sigma_xyz = np.sqrt(beta**2 * sigma_spy**2 + sigma_eps**2)
```

```
print(f"Return days: {len(returns)}  |  last XYZ price: {prices['XYZ']
display(pd.Series({
    "alpha": alpha,
    "beta": beta,
    "mu_SPY": mu_spy,
    "sigma_SPY": sigma_spy,
    "sigma_epsilon": sigma_eps,
    "mu_XYZ": mu_xyz,
    "sigma_XYZ": sigma_xyz,
}, name="estimate").to_frame())

def normal_var(mu, sigma, alpha):
    """Positive loss VaR: -F^{-1}(alpha) for a N(mu, sigma^2) return."
    return -(mu + sigma * stats.norm.ppf(alpha))

var_table = pd.DataFrame([
    {"level": level, "VaR (return)": normal_var(mu_xyz, sigma_xyz, a),
     "VaR (%)": 100 * normal_var(mu_xyz, sigma_xyz, a)}
    for level, a in [("95%", 0.05), ("99%", 0.01)]
]).set_index("level")
print("One-day VaR, percent of the XYZ position:")
display(var_table.round(6))
```

```
Return days: 500  |  last XYZ price: 74.5183
```

	estimate
alpha	-3.040218e-04
beta	1.193311e+00
mu_SPY	2.547717e-04
sigma_SPY	1.097207e-02
sigma_epsilon	1.459005e-02
mu_XYZ	2.602085e-18
sigma_XYZ	1.960353e-02

```
One-day VaR, percent of the XYZ position:
```

level	VaR (return)	VaR (%)
95%	0.032245	3.224494
99%	0.045605	4.560464

$\beta \approx 1.193$. The fitted mean of r_{XYZ} is numerically zero, so $VaR_\alpha \approx \sigma|z_\alpha|$.

95% VaR = 3.2245% and **99% VaR = 4.5605%** of the position.

(b) ES

$$ES(X) = E(X \mid X \leq -VaR(X))$$

X is the return, so this expectation is negative. The loss we report is the positive number $-ES(X)$, in percent (Week 05). For the normal model from (a),

$$-E[r \mid r \leq -VaR_\alpha] = -\mu + \sigma \frac{\phi(z_\alpha)}{\alpha}.$$

In [8]:
```
# Problem 2(b)
def normal_es(mu, sigma, alpha):
    """Positive loss ES of a N(mu, sigma^2) return."""
    z = stats.norm.ppf(alpha)
    return -(mu - sigma * stats.norm.pdf(z) / alpha)

es_table = pd.DataFrame([
    {"level": level,
     "VaR (%)": 100 * normal_var(mu_xyz, sigma_xyz, a),
     "ES (return)": normal_es(mu_xyz, sigma_xyz, a),
     "ES (%)": 100 * normal_es(mu_xyz, sigma_xyz, a)}
    for level, a in [("95%", 0.05), ("99%", 0.01)]
]).set_index("level")
print("One-day ES, percent of the XYZ position:")
display(es_table.round(6))
```

```
One-day ES, percent of the XYZ position:
```

level	VaR (%)	ES (return)	ES (%)
95%	3.224494	0.040436	4.043646
99%	4.560464	0.052248	5.224762

95% ES = 4.0436% and **99% ES = 5.2248%** of the position. ES is larger than VaR at the same level because it is the average loss beyond the VaR cutoff.

Question 3 — Copula VaR and ES for a Two-Book Portfolio

(a) Normal, Student's t, NIG for Return

Fit three distributions to each return series and keep the one with the lowest AICc (from Week 02's note 2.5.3; lower is better):

$$\mathrm{AICc} = -2\ell + 2k + \frac{2k(k+1)}{n-k-1}.$$

- **Normal.** Sample mean and sample standard deviation ($ddof = 1$), as in `fit_normal`. $k = 2$.
- **Student's t.** MLE of $(\nu,\ \mathrm{loc},\ \mathrm{scale})$, as in `fit_general_t`. $k = 3$. A symmetric t has heavy tails and no skew.
- **Normal inverse Gaussian.** MLE from `scipy.stats.norminvgauss`, converted to the Week 01 parameters $(\mu, \alpha, \beta, \delta)$ by $\delta = \mathrm{scale}$, $\alpha = a/\delta$, $\beta = b/\delta$, $\mu = \mathrm{loc}$. $k = 4$. β is the skew.

The t is the right extra step when the tails are heavy and the sample is nearly symmetric. The NIG is the right extra step when the sample is also skewed. AICc drops a model whose extra parameters do not pay for themselves.

In [9]:
```
# Problem 3(a)
import warnings
from scipy import stats

warnings.filterwarnings("ignore")

prices = pd.read_csv("problem3.csv", parse_dates=["Date"]).set_index("
holdings = pd.read_csv("problem3_holdings.csv").set_index("Stock")
returns = prices.pct_change().dropna()
names = list(returns.columns)
n_obs = len(returns)


def aicc(ll, k, n):
    return -2 * ll + 2 * k + 2 * k * (k + 1) / (n - k - 1)


fits = {}
rows = []
for stock in names:
    x = returns[stock].to_numpy()
    mu, sigma = float(x.mean()), float(x.std(ddof=1))
    ll_n = float(stats.norm.logpdf(x, mu, sigma).sum())
    df, loc, scale = (float(v) for v in stats.t.fit(x))
    ll_t = float(stats.t.logpdf(x, df, loc, scale).sum())
    a, b, loc_n, scale_n = (float(v) for v in stats.norminvgauss.fit(x
    ll_g = float(stats.norminvgauss.logpdf(x, a, b, loc_n, scale_n).su
    delta, alpha, beta, mu_n = scale_n, a / scale_n, b / scale_n, loc_
    scores = {
        "Normal": aicc(ll_n, 2, n_obs),
        "Student-t": aicc(ll_t, 3, n_obs),
        "NIG": aicc(ll_g, 4, n_obs),
    }
    best = min(scores, key=scores.get)
```

```
    fits[stock] = {
        "best": best,
        "normal": (mu, sigma),
        "t": (df, loc, scale),
        "nig_scipy": (a, b, loc_n, scale_n),
        "nig": (mu_n, alpha, beta, delta),
    }
    rows.append({
        "Stock": stock,
        "skew": float(pd.Series(x).skew()),
        "excess kurtosis": float(pd.Series(x).kurtosis()),
        "chosen": best,
        "AICc Normal": scores["Normal"],
        "AICc t": scores["Student-t"],
        "AICc NIG": scores["NIG"],
        "mu": mu, "sigma": sigma,
        "t df": df, "t loc": loc, "t scale": scale,
        "NIG mu": mu_n, "NIG alpha": alpha, "NIG beta": beta, "NIG del
    })

fit_table = pd.DataFrame(rows).set_index("Stock")
print(f"Return days: {n_obs}. Chosen models:", fit_table["chosen"].val
display(fit_table.round(4))
```

```
Return days: 1000. Chosen models: {'Student-t': 4, 'Normal': 3, 'NIG':
3}
```

Stock	skew	excess kurtosis	chosen	AICc Normal	AICc t	...	t scale	NIG mu
S1	-0.0397	-0.0267	Normal	-6033.9932	-6031.9817	...	0.0118	0.0766
S2	0.0707	5.5107	Student-t	-5512.7054	-5627.3597	...	0.0117	-0.0003
S3	-1.3228	4.7260	NIG	-5096.5124	-5238.4631	...	0.0137	0.0105
S4	-0.0347	-0.1273	Normal	-6382.9735	-6380.9619	...	0.0099	0.0895
S5	-0.3079	2.7523	Student-t	-5090.8947	-5150.8695	...	0.0156	0.0015
S6	-1.2039	4.7292	NIG	-5547.2030	-5688.7400	...	0.0107	0.0083
S7	-0.0407	-0.1178	Normal	-5743.6938	-5741.6820	...	0.0137	0.0775
S8	-0.4446	6.5676	Student-t	-5295.1064	-5493.2876	...	0.0114	0.0012
S9	-1.0849	4.4898	NIG	-4919.5057	-5036.7625	...	0.0153	0.0101
S10	-0.6974	5.6166	Student-t	-5835.3242	-5974.8570	...	0.0094	0.0010

10 rows × 15 columns

Normal for S1, S4, and S7. Their fitted t degrees of freedom run into the hundreds of thousands, so the t has collapsed to the normal and AICc rejects the extra parameter. Skew and excess kurtosis are small.

Student's t for S2, S5, S8, and S10. Excess kurtosis is large and skew is mild, which is what a symmetric heavy-tailed margin is for. ν is about 3.5 to 6.3.

NIG for S3, S6, and S9. Those series are skewed as well as heavy-tailed, and the NIG β is clearly nonzero. The four-parameter penalty is paid for by the likelihood.

(b) Gaussian Copula and Student's t Copula

Use the marginal chosen in (a). Transform each series to $U_i = F_i(x_i)$. Both copulas get the same correlation, built from Kendall's τ (Week 05, 5.5.3.1):

$$\rho_{ij} = \sin(\pi\tau_{ij}/2).$$

That inversion is exact for an elliptical copula, so a gap in likelihood is the copula family and not a different R. Repair R only if an eigenvalue is negative.

The copula log likelihood is the joint log density of the transformed variables minus the sum of their marginal log densities (`gaussian_copula_ll` and `t_copula_ll`). Profile ν on $\theta = 1/\nu \in [0.01, 0.49]$, the same grid-and-refine search as `profile_nu` .

R is held fixed, so $k_{\text{Gauss}} = 0$ and $k_t = 1$. Take the t copula when

$$\Delta\text{BIC} = 2(\ell_t - \ell_{\text{Gauss}}) - \ln m > 0.$$

Report AICc as well. The two agree here.

In [21]:
```
# Problem 3(b)
def fix_correlation(R, tol=1e-8, ridge=1e-8):
    """Symmetrize. If an eigenvalue is negative, repair with Higham."""
    R = 0.5 * (np.asarray(R, float) + np.asarray(R, float).T)
    if np.linalg.eigvalsh(R).min() < -tol:
        R, _ = higham_nearest_correlation(R)
    if np.linalg.eigvalsh(R).min() < ridge:
        R = (R + ridge * np.eye(len(R))) / (1.0 + ridge)
    return 0.5 * (R + R.T)


def gaussian_copula_ll(U, R):
    Z = stats.norm.ppf(U)
    joint = stats.multivariate_normal.logpdf(Z, mean=np.zeros(R.shape[
    return float(joint - stats.norm.logpdf(Z).sum())


def t_copula_ll(U, R, nu):
    T = stats.t.ppf(U, df=nu)
    joint = stats.multivariate_t.logpdf(T, loc=np.zeros(R.shape[0]), s
    return float(joint - stats.t.logpdf(T, df=nu).sum())


def profile_nu(loglik, lo=0.01, hi=0.49, n_grid=100):
    thetas = np.linspace(lo, hi, n_grid)
    values = np.array([loglik(1.0 / th) for th in thetas])
    i = int(np.argmax(values))
    fine = np.linspace(thetas[max(i - 1, 0)], thetas[min(i + 1, n_grid
    fine_values = np.array([loglik(1.0 / th) for th in fine])
    j = int(np.argmax(fine_values))
    return float(1.0 / fine[j]), float(fine_values[j])


U = np.column_stack([
    stats.norm.cdf(returns[s], *fits[s]["normal"]) if fits[s]["best"]
    else stats.t.cdf(returns[s], *fits[s]["t"]) if fits[s]["best"] ==
    else stats.norminvgauss.cdf(returns[s], *fits[s]["nig_scipy"])
    for s in names
])
U = np.clip(U, 1e-12, 1 - 1e-12)
```

```
tau = pd.DataFrame(U, columns=names).corr(method="kendall").to_numpy()
R_tau = np.sin(np.pi * tau / 2.0)
np.fill_diagonal(R_tau, 1.0)
R_tau = fix_correlation(R_tau)
# Spearman of U is the Gaussian-copula correlation from the Week 05 fi
R_spearman = fix_correlation(pd.DataFrame(U, columns=names).corr(metho

nu_hat, ll_t = profile_nu(lambda nu: t_copula_ll(U, R_tau, nu))
ll_g = gaussian_copula_ll(U, R_tau)
m = n_obs


def copula_aicc(ll, k):
    return -2 * ll + 2 * k + (2 * k * k + 2 * k) / (m - k - 1)


def copula_bic(ll, k):
    return k * np.log(m) - 2 * ll


delta_bic = 2 * (ll_t - ll_g) - np.log(m)
copula_choice = "Student-t" if delta_bic > 0 else "Gaussian"
copula_table = pd.DataFrame({
    "Gaussian": [ll_g, 0, np.nan, copula_aicc(ll_g, 0), copula_bic(ll_
    "Student-t": [ll_t, 1, nu_hat, copula_aicc(ll_t, 1), copula_bic(ll
}, index=["log likelihood", "free parameters", "fitted nu", "AICc", "B
print(f"Kendall R min eigenvalue: {np.linalg.eigvalsh(R_tau).min():.4f
print(f"Spearman R min eigenvalue: {np.linalg.eigvalsh(R_spearman).min
print(f"||R_spearman - R_tau||_F = {np.linalg.norm(R_spearman - R_tau)
display(copula_table.round(2))
print(f"delta BIC = {delta_bic:.2f}  -->  choose the {copula_choice} c
```

```
Kendall R min eigenvalue: 0.3820
Spearman R min eigenvalue: 0.4066
||R_spearman - R_tau||_F = 0.1506
```

	Gaussian	Student-t
log likelihood	2110.02	2108.16
free parameters	0.00	1.00
fitted nu	NaN	100.00
AICc	-4220.04	-4214.32
BIC	-4220.04	-4209.42

```
delta BIC = -10.63  -->  choose the Gaussian copula
```

The profile for ν sits on the upper edge of the search ($\nu = 100$), and the t log likelihood there is still below the Gaussian. $\Delta\text{BIC} \approx -10.6 < 0$, and AICc ranks the two models the same way, so the Gaussian copula fits better.

A large ν is the Gaussian copula. The data do not pay for a finite-ν joint tail. Under the Gaussian copula, lower tail dependence is 0 at every correlation below 1.

(c) Portfolio Simulation and absolute VaR and ES.

The Gaussian copula won, so the production correlation is the Spearman matrix of U (Week 05: compare the families on Kendall's τ, then use Spearman for the Gaussian simulation). Draw 10,000 (>5000) standard normals with that correlation, map them through Φ to uniforms, and map each uniform through the marginal quantile from (a).

Dollar P&L of stock i on a draw is $h_i P_{i,T} r_i$, with $P_{i,T}$ the last price. Book and portfolio P&L are sums of those stock P&Ls on the **same** draw.

VaR and ES use the rule at $\alpha = 0.05$. Sort the P&L, average the order statistics at $\lfloor n\alpha \rfloor$ and $\lceil n\alpha \rceil$, and take ES as the mean of every P&L at or below that cutoff. Both are reported as a positive dollar loss.

In [ ]:
```
# Problem 3(c)
def marginal_ppf(stock):
    """Quantile of the marginal chosen in (a). NIG has no closed form,
    fit = fits[stock]
    if fit["best"] == "Normal":
        mu, sigma = fit["normal"]
        return lambda u: stats.norm.ppf(u, mu, sigma)
    if fit["best"] == "Student-t":
        df, loc, scale = fit["t"]
        return lambda u: stats.t.ppf(u, df, loc, scale)
    a, b, loc_n, scale_n = fit["nig_scipy"]
    lo = float(stats.norminvgauss.ppf(1e-6, a, b, loc_n, scale_n))
    hi = float(stats.norminvgauss.ppf(1 - 1e-6, a, b, loc_n, scale_n))
    grid = np.linspace(lo, hi, 6000)
    cdf = np.maximum.accumulate(stats.norminvgauss.cdf(grid, a, b, loc
    return lambda u, cdf=cdf, grid=grid: np.interp(np.clip(u, cdf[0],


def var_es(pnl, alpha=0.05):
    """Positive-loss VaR and ES. Same order-statistic rule defined in
    x = np.sort(np.asarray(pnl, float))
    n_up = int(np.ceil(len(x) * alpha))
    n_dn = int(np.floor(len(x) * alpha))
    cutoff = 0.5 * (x[n_up - 1] + x[n_dn - 1])
    return float(-cutoff), float(-x[x <= cutoff].mean())


N_SIM = 10_000
```

```
rng = np.random.default_rng(1234)
R_sim = R_tau if copula_choice == "Student-t" else R_spearman
Z = rng.standard_normal((N_SIM, len(names))) @ np.linalg.cholesky(R_si
if copula_choice == "Student-t":
    shock = nu_hat / rng.chisquare(nu_hat, size=N_SIM)
    U_sim = stats.t.cdf(np.sqrt(shock)[:, None] * Z, df=nu_hat)
else:
    U_sim = stats.norm.cdf(Z)
U_sim = np.clip(U_sim, 1e-8, 1 - 1e-8)

simulated = np.column_stack([marginal_ppf(stock)(U_sim[:, j]) for j, s
exposure = (holdings.loc[names, "Holding"] * prices.iloc[-1][names]).a
books = holdings.loc[names, "Book"]
pnl = simulated * exposure.to_numpy()[None, :]

risk_rows = []
for j, stock in enumerate(names):
    var, es = var_es(pnl[:, j])
    risk_rows.append({
        "Position": stock, "Book": books[stock], "Model": fits[stock][
        "Value ($)": float(exposure[stock]), "95% VaR ($)": var, "95%
    })
for book in ["A", "B"]:
    mask = (books == book).to_numpy()
    var, es = var_es(pnl[:, mask].sum(axis=1))
    risk_rows.append({
        "Position": f"Book {book}", "Book": book, "Model": "",
        "Value ($)": float(exposure[mask].sum()), "95% VaR ($)": var,
    })
var, es = var_es(pnl.sum(axis=1))
risk_rows.append({
    "Position": "Total", "Book": "A+B", "Model": copula_choice + " cop
    "Value ($)": float(exposure.sum()), "95% VaR ($)": var, "95% ES ($
})
risk_table = pd.DataFrame(risk_rows).set_index("Position")
print(f"{N_SIM:,} draws, seed 1234, {copula_choice} copula, Spearman c
display(risk_table.style.format({
    "Value ($)": "${:,.2f}", "95% VaR ($)": "${:,.2f}", "95% ES ($)":
}))
```

```
10,000 draws, seed 1234, Gaussian copula, Spearman correlation of U
```

Position	Book	Model	Value ($)	95% VaR ($)	95% ES ($)
S1	A	Normal	$48,329.48	$887.23	$1,117.17
S2	A	Student-t	$46,697.43	$1,057.42	$1,525.69
S3	A	NIG	$24,267.28	$786.14	$1,198.24
S4	A	Normal	$43,596.81	$678.98	$853.92
S5	A	Student-t	$37,727.45	$1,096.60	$1,532.75
S6	B	NIG	$46,326.64	$1,205.19	$1,803.03
S7	B	Normal	$40,144.31	$888.86	$1,120.79
S8	B	Student-t	$26,045.88	$644.03	$1,069.67
S9	B	NIG	$98,248.71	$3,433.71	$5,201.04
S10	B	Student-t	$24,512.38	$486.94	$720.36
Book A	A		$200,618.44	$3,513.33	$4,747.20
Book B	B		$235,277.92	$5,484.06	$7,914.08
Total	A+B	Gaussian copula	$435,896.36	$7,720.23	$10,680.14

Discussion, part (c)

The one-day 95% loss on the whole book is **VaR $7,720** and **ES $10,680**, on a portfolio worth $435,896. ES is larger than VaR because it averages the losses past the cutoff rather than stopping at it.

Position	Value	95% VaR	95% ES
Book A	$200,618	$3,513	$4,747
Book B	$235,278	$5,484	$7,914
Total	$435,896	$7,720	$10,680

Book B is the larger risk even after its larger value is taken into account. S9 alone is a $98,249 NIG position with a 95 3,434, more than half of Book B's VaR. Book A's five names are closer in size, and three of them are nearly normal.

Adding the two books is a real diversification. Standalone VaRs sum to about $8,997$ against a combined VaR of 7,720 (benefit about $1,277$). The ES benefit is about 1,981. The books are not independent — the copula correlation is positive — so the benefit is only a fraction of the standalone

risk.

Two limits belong with the number. The Gaussian copula has zero tail dependence, so it will understate a day on which every name crashes together (like the 08 financial crisis where we mentioned in class); the likelihood said that extra tail dependence was not in this sample. The exposures are the last prices times the given shares, with no rebalancing inside the day.